Proceedings of the ASME 2026
International Design Engineering Technical Conferences &
Computers and Information in Engineering Conference
IDETC-CIE 2026
August 23–26, 2026, Houston, TX, USA

DETC2026-XXXX

NATURAL-LANGUAGE TO SYSMLV2 TRANSLATION VIA VALIDATOR-DRIVEN ITERATIVE REFINEMENT

Chance LaVoie[1], Eladio Andujar Lugo[1], Taylan G. Topcu[2], Levent Burak Kara[1,*]

[1]Department of Mechanical Engineering, Carnegie Mellon University, Pittsburgh, PA 15213 USA
[2]Grado Department of Industrial and Systems Engineering, Virginia Tech, Blacksburg, VA 24061 USA

ABSTRACT

Model-Based Systems Engineering (MBSE) relies on formal SysMLv2 models as primary technical artifacts for representing requirements, structure, and behavior across the system lifecycle. With the standardization of SysMLv2 as a textual language, there is growing interest in translating natural-language engineering descriptions directly into executable models. However, for practical deployment, generated models must be accepted by industrial modeling environments, not merely satisfy context-free grammar constraints. This paper presents a validator-driven framework for reliable natural-language–to–SysMLv2 translation that enforces production-level acceptance as a termination condition. The proposed system embeds a production SysMLv2 validator within a generate–validate–repair loop. Each candidate model is evaluated using a production compiler, and deterministic diagnostics are incorporated into subsequent revisions until zero validation errors are achieved. By using the production validator as the correctness oracle, the framework targets operational deployability rather than grammar plausibility alone. We evaluate the approach on the full SysMBench prompt set comprising 151 prompts across four large language model backends, yielding 604 prompt–model cases. Single-shot generation achieves 51.16% production-validation acceptance, whereas validator-guided refinement achieves 100.00% acceptance across all cases. Convergence is rapid, with most recoverable failures resolved within a small number of repair cycles. By elevating production validation from a post-processing check to a control mechanism within generation, this work converts probabilistic text outputs into validator-accepted SysMLv2 artifacts suitable for loading, visualization, and downstream engineering use. To support reproducibility and further research, we release a trajectory-level corpus of all refinement traces.

Keywords: Systems Engineering, Design Automation, AI/KBS, Software Agents/Systems, Design Representation

*Corresponding author: lkara@andrew.cmu.edu

1. INTRODUCTION

Model-Based Systems Engineering (MBSE) replaces document-centric workflows with model-centric engineering, where formal artifacts represent requirements, structure, behavior, and verification intent across the system lifecycle [1, 2]. By elevating the model to the primary technical authority, MBSE reduces cross-document reconciliation effort and improves traceability under change [2, 3]. SysMLv2 strengthens this transition by standardizing a formal language with complementary textual and graphical representations and machine-readable artifacts under the Object Management Group [4]. In practice, engineers can author the same model textually, render it visually, and exchange it across tools with reduced ambiguity. Because SysMLv2 is formal and textual, model construction is inherently scriptable. This scriptability creates an opportunity to translate natural-language engineering descriptions directly into executable SysMLv2 artifacts. However, for such translation to be useful in practice, generated models must be accepted by industrial modeling environments, not merely resemble well-formed text.

Recent advances in large language models (LLMs) suggest that automated generation can meaningfully accelerate engineering workflows. In software engineering, controlled and field studies report substantial productivity gains from LLM-assisted generation, including 55.8% faster task completion, 26.08% higher weekly completed tasks in enterprise randomized deployments, and measurable increases in pull-request throughput [5–7]. These results motivate analogous integration in MBSE, where automating low-value authoring steps could increase modeling speed and allow teams to focus on architectural and verification reasoning.

Current State in MBSE: Despite this opportunity, LLMs do not reliably produce production-validator-accepted[1] SysMLv2 models in single-shot generation. In response to this shortcom-

[1]Production-validator-accepted: accepted by the production SysMLv2 validator with zero reported errors, indicating tool-level operational usability rather than grammar validity alone.

Copyright ©2026 by ASME

ing, recent work has proposed structured pipelines for natural-language–to–SysMLv2 generation [8, 9]. SysTemp employs a template-first multi-agent design in which a template generator constructs a structured skeleton and a parser agent iteratively corrects syntax using grammar feedback [8]. Cibrián et al. organize generation as an agentic loop with a retrieval-augmented context engine and ANTLR-based grammar validation as the primary syntax gate [9]. In parallel, SysMBench establishes a benchmark for evaluating natural-language–to–SysML generation quality, especially with respect to semantic alignment [10]. Collectively, these efforts demonstrate that grammar-conforming SysMLv2 can be produced from natural language.

Limits of Grammar-Level Validation: Despite this progress, grammar-level validity does not guarantee operational usability. Context-free grammar parsing ensures correct token ordering, balanced delimiters, and production-rule-compliant declarations. Industrial validators enforce additional model-wide constraints, including name resolution, type consistency, ownership rules, multiplicity constraints, and cross-reference integrity. A model that passes ANTLR parsing can still fail production validation and therefore remain unusable for visualization, simulation, or downstream analysis. In an auxiliary repository demonstration, highlighted in Appendix A, we show ten distinct examples that pass SysML ANTLR parsing yet fail production validation [11]. The gap is therefore not grammar feasibility, but reliable generation of production-validator–accepted SysMLv2 from arbitrary natural-language prompts.

Proposed advance: In this work, we present a validator-driven framework for natural-language–to–SysMLv2 generation that requires every generated model to pass a production validator before it is considered complete. We place a production SysMLv2 validator, SysIDE [12], inside a generate–validate–refine loop. Given a natural-language prompt, the system produces a full textual SysMLv2 model and immediately checks it using the production validator. The reported diagnostics, such as unresolved references, typing errors, and ownership violations, are then used to revise the model. We repeat this process until the validator reports zero errors.

This approach follows the general idea of oracle-guided refinement from formal methods and inductive synthesis [13–16], and is similar in spirit to compiler-feedback-driven code generation [17, 18]. The difference here is that the oracle is a production modeling tool rather than a grammar parser. As a result, the loop enforces the same model-wide checks that an engineer would encounter when loading the model into an industrial environment.

We evaluate the framework on the full SysMBench prompt set of 151 prompts across four model backends, yielding 604 prompt–model cases. In single-shot generation, 51.16% of the initial models pass production validation. With validator-guided refinement, all 604 cases reach zero-error acceptance, with most resolving in only a few repair cycles. These results show that production-validator feedback provides a reliable signal for correcting generated SysMLv2 models and makes direct integration into MBSE workflows feasible.

Our main contributions are:

1. A validator-in-the-loop architecture for natural-language–to–SysMLv2 translation that enforces production-validator acceptance as a termination invariant.
2. A benchmark-scale empirical study across the full SysMBench prompt set and four model backends that quantifies the gap between grammar validity and production acceptance.
3. A trajectory-level corpus of validator-guided refinement traces that enables future research on reliability, repair dynamics, and deployment-scale NL-to-MBSE systems.

2. RELATED WORK

Our review focuses on three areas that intersect in this study: (1) natural-language–to–SysML model generation within MBSE, (2) grammar-constrained and structured synthesis methods for large language models, and (3) verifier- and tool-guided refinement loops for generated artifacts. These bodies of work collectively demonstrate the feasibility of structured model synthesis from natural language and the benefits of deterministic feedback, while also highlighting open questions regarding deployment-level acceptance in industrial modeling environments.

2.1. Natural-Language to SysML Model Generation

Recent research has explored the use of large language models to generate SysML and SysMLv2 artifacts directly from natural-language requirements. SysTemp [8] introduces a template-driven, multi-agent pipeline in which a structured model skeleton is first constructed and then refined through parser-guided correction. Cibrián et al. [9] propose an agent-based architecture that combines retrieval-augmented generation with ANTLR-based grammar validation to improve syntactic correctness. These systems demonstrate that structured prompting and iterative repair can substantially improve parser-level validity for SysMLv2 text.

Additional studies examine LLM behavior in SysML-related tasks more broadly. Wang et al. [19] report empirical findings on the generation of SysML behavioral models and identify common inconsistencies and hallucinations in model structure. Other work investigates LLM-assisted interaction with SysMLv2 artifacts in engineering settings [20] and describes workflows in which generative AI is used to build and query MBSE models**CITATION?** [**?**]. These contributions reflect a growing interest in integrating LLMs into model-centric engineering practice.

Earlier efforts relied on restricted natural-language subsets, rule-based extraction, or diagram-level heuristics to translate requirements into SysML models [21–23]. While effective in constrained settings, these approaches often required controlled vocabularies or domain-specific grammars. More recent work explores LLM-assisted semantic alignment and model integration across SysMLv2 artifacts [24, 25]. Collectively, this literature establishes the technical feasibility of NL-to-SysML pipelines, but most evaluations emphasize grammar conformity or diagram-level correctness rather than production-tool acceptance. Our work complements these efforts by examining acceptance under a production validation backend as the primary reliability criterion.

Copyright ©2026 by ASME

2.2. Grammar-Constrained and Structured Synthesis

Grammar-constrained decoding and template-based generation have emerged as practical strategies for improving structural correctness in LLM outputs. Grammar-constrained decoding methods restrict token sequences to context-free grammars, thereby enforcing well-formed structural patterns without requiring model retraining [26–28]. These approaches reduce malformed outputs and improve syntactic validity across a range of structured generation tasks.

Template-driven methods further guide generation by providing predefined structural scaffolds or repair templates**CITATION?** [**?**]. In SysML contexts, template-first pipelines such as SysTemp [8] use structured model outlines to stabilize generation. Such methods are particularly useful for newly standardized or low-data languages.

While grammar and template constraints effectively address token-level validity, they do not necessarily enforce model-wide consistency or tool-specific constraints. Production modeling environments typically impose additional checks related to reference resolution, typing consistency, and cross-model integrity. As a result, grammar-level correctness represents an important but partial notion of validity. Our approach builds on structured synthesis methods while shifting the acceptance criterion from parser conformity to production-tool validation.

2.3. Verifier- and Tool-Guided Refinement

Iterative refinement using deterministic feedback has a long history in formal methods and program synthesis. Counterexample-guided abstraction refinement and oracle-guided synthesis frameworks use external checkers to iteratively improve candidate artifacts [13, 15, 16]. In the context of large language models, Wang et al. [17] and Grubišić et al. [18] demonstrate that compiler diagnostics can serve as structured correction signals for generated code. Subsequent work extends this paradigm to test-driven and verifier-driven loops [29–32].

Across these domains, a consistent pattern emerges: candidate artifacts are generated, evaluated by a deterministic backend, and revised until the backend reports success. In software engineering, the oracle is typically a compiler, test suite, or formal verifier. Our work instantiates this refinement pattern in the context of SysMLv2 MBSE, where the oracle is a production modeling tool. By aligning the termination condition with the same validation mechanism used in industrial environments, we extend verifier-guided refinement to model-based systems engineering and examine its effect at benchmark scale.

3. METHODOLOGY

In this section, we describe the experimental design and the validator-in-the-loop procedure used to evaluate natural-language–to–SysMLv2 generation. We first define the paired study setup, and then detail the iterative generation and validation workflow.

3.1. Study Objective and Paired Design

Our goal is to evaluate whether placing a production validator inside the generation loop improves reliability relative to single-shot generation. Specifically, we ask: for the same natural-language prompt and the same language model, does iterative validator-guided refinement increase the rate of production validation acceptance? In this study, we focus strictly on syntactic and production-level validation.

The experimental unit is a single prompt–model pair. For each pair, we compare two conditions obtained from the same generation trajectory:

1. **Baseline (single-shot):** the production-validation outcome of the initial model produced by the language model ($k = 0$).
2. **Pipeline (iterative):** the production-validation outcome after applying validator-guided repair until termination.

Because both outcomes are derived from the same prompt and the same model instance, this paired design isolates the effect of iterative validator feedback. Prompt content and model identity remain fixed, and only the presence or absence of validator-driven refinement differs between conditions.

3.2. Validator-in-the-Loop Generation Procedure

We implement a generate–validate–repair workflow for natural-language–to–SysMLv2 translation. Given a prompt, the language model produces a complete textual SysMLv2 candidate. We then evaluate this candidate using a production validator. If validation errors are reported, we pass the deterministic diagnostics back to the model and request a revised candidate. This process continues until the validator reports zero errors.

Let P denote the natural-language prompt, M_t the SysMLv2 model generated at iteration t, and $V(\cdot)$ the production validator. The update at each iteration can be written as

$$M_{t+1} = f(P, M_t, V(M_t)),$$

where $f(\cdot)$ represents the model call conditioned on the prompt, the previous candidate, and the validator diagnostics. In practice, $V(M_t)$ returns structured error messages that identify unresolved references, typing inconsistencies, ownership violations, and other model-wide constraint failures. These diagnostics guide the subsequent revision.

We use SysIDE (`syside check`) as the production validation oracle [12]. A run terminates only when the validator reports zero errors. This choice is motivated by the distinction between grammar-level parsing and production acceptance. In an auxiliary ten-case study included in our repository and appendix, we demonstrate that models can pass the HAMR SysML v2 ANTLR4 parser while still failing production validation, illustrating that grammar conformity alone does not ensure operational usability [11, 33, 34]. By using the production validator as the acceptance criterion, we align the loop with the same checks encountered in an industrial modeling environment.

Copyright ©2026 by ASME

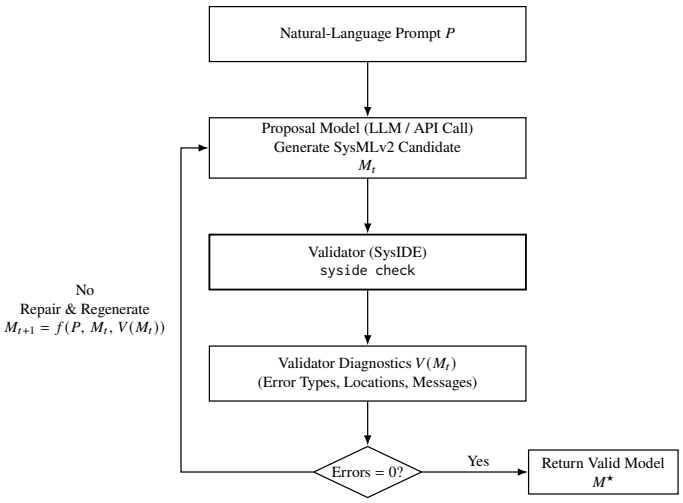

FIGURE 1: Validator-in-the-loop generate–validate–repair workflow. The prompt remains fixed for each case, and each revision is driven by deterministic validator diagnostics.

3.3. Dataset and Outcome Extraction

SysMBench provides paired natural-language prompts and ground-truth SysMLv2 models for benchmark evaluation [10]. In this study, we use the curated natural-language prompt set (IDs 1–151) as generation inputs because it was designed to stress SysMLv2 LLM generation across diverse modeling patterns.

To demonstrate the variety and structure of the SysMBench prompts, Figure 2 shows three representative prompt excerpts demonstrating a compact photography workflow specification, a spatiotemporal vehicle traffic simulation prompt, and a safety-critical medical health monitoring prompt.

To assess model-agnostic behavior of the same controller, we run four model configurations: OpenAI Codex 5.2 (`gpt-5.2-codex`) [35], Anthropic Sonnet 4.6 (`claude-sonnet-4-6`) [36], DeepSeek Reasoner (`deepseek-reasoner`) [37], and Mistral Large (`mistral-large-latest`) [38]. This yields 604 prompt-level cases (151 prompts $\times$ 4 models).

From saved run records, we extract single-shot and pipeline pass/fail outcomes, iterations run, iterations to success, first/final error counts, cumulative error counts, per-iteration runtime, and token usage. For grammar-versus-production analysis, we also extract single-shot ANTLR parse pass/fail and single-shot SysIDE pass/fail from the same initial candidate artifacts for all 604 prompt–model cases. Error families are grouped from validator diagnostics (for example, parsing and reference errors), while warnings are tracked separately and do not change pass/fail labels.

3.4. Evaluation Metrics

3.4.1. Production Validation Acceptance The primary evaluation metrics are production validation acceptance rates. For each analysis slice (overall and per-model), we report:

1. single-shot pass rate (initial candidate, $k = 0$),
2. pipeline pass rate (final iteration of the validator-gated loop).

We also report iterations-to-success, defined as the number of repair cycles required for a prompt–model case to reach production-validator acceptance.

3.4.2. Grammar vs Production Validity To quantify the operational gap between grammar-level validity and production-validator acceptance, we compare ANTLR and SysIDE outcomes on the same single-shot artifact from each case. This comparison is performed on the initial candidate ($k = 0$ in repair-cycle indexing).

Grammar validity is defined as ANTLR parse success using the HAMR SysML v2 ANTLR4 parser, which provides a generated grammar and Java parser/lexer derived from the SysML v2 Pilot Implementation [33, 34]. This ANTLR check is used only as a representative context-free grammar (CFG) parsability test and does not enforce production model-wide constraints. Production validity is defined as SysIDE acceptance (zero SysIDE errors) under `syside check`. For each case, we record binary pass/fail outcomes for both checks and construct a 2×2 contingency table (ANTLR pass/fail $\times$ SysIDE pass/fail).

Compact Workflow Prompt Prompt 85

Photography Technique

The system is designed to implement a camera information processing workflow. When a user selects a scene through the camera's viewfinder (viewPort), the system first focuses on the scene to obtain an image (Image). This image is then captured to generate a photograph (Picture). After the photograph is generated, the system displays it on the screen via the display port (displayPort).

Spatiotemporal Simulation Prompt Prompt 26

Vehicle Traffic

This system is designed for spatiotemporal simulation of the dynamic behavior of vehicles on roads at different moments. Users can define parameters such as the vehicle's mass, position, velocity, and acceleration, and, combined with the road's slope (angle) and surface friction coefficient, depict the state of the vehicle and the road at specific time points. The system supports snapshot recording at multiple moments within the simulation time series, enabling tracking of the vehicle's state transitions from start-up (on state), through the driving process, to shutdown (off state).

Safety-Critical Monitoring Prompt Prompt 48

Medical Health

This system is designed to ensure high reliability and safety of the blood glucose meter during use. When the battery is depleted or cannot be charged, the system should be able to automatically detect the battery status and promptly alert the user to prevent failure to measure blood glucose levels due to battery issues, as well as potential treatment delays resulting from such failures. To prevent the aforementioned failure scenarios, the system requires the implementation of preventive measures for battery status, and it must have appropriate alarm and emergency response mechanisms in case of abnormalities in the blood glucose measurement function.

FIGURE 2: Representative SysMBench prompt excerpts illustrating the variety and structure of the benchmark inputs. The examples show a compact workflow prompt (Prompt 85), a spatiotemporal vehicle-simulation prompt (Prompt 26), and a safety-critical monitoring prompt (Prompt 48).

Copyright ©2026 by ASME

3.4.3. Convergence Metrics To characterize how quickly the loop approaches acceptance, we index progress by repair cycles. Let $k = 0$ denote the initial single-shot generation (no validator feedback), and let $k \geq 1$ denote k rounds of generate–validate–repair. Let A_k denote cumulative production-validation acceptance after repair cycle k.

We define residual failure mass as

$$R_k = 1 - A_k,$$

where R_k is the fraction of prompt–model cases not yet accepted at repair cycle k. We then define an empirical contraction ratio

$$\rho_k = \frac{R_{k+1}}{R_k}.$$

When $0 < \rho_k < 1$, residual failure shrinks from one cycle to the next; when ρ_k is approximately stable across k, the observed trajectory suggests contraction-like behavior, with approximately multiplicative reduction in the early repair regime under standard contraction-style analyses in iterative methods [39–41]. In finite empirical campaigns, we estimate contraction behavior from early-to-mid cycles, before residual mass becomes very small; terminal transitions are excluded from rate estimation because finite-sample tail effects can produce unstable ratios near the finite-sample resolution. This characterization is descriptive of observed finite-sample behavior and does not assert formal convergence guarantees.

We also report time-to-threshold acceptance

$$T_\varepsilon = \min\{k : A_k \geq 1 - \varepsilon\},$$

with explicit reporting of T_{90}, T_{95}, and T_{99}. This follows standard iteration-to-threshold (iteration-to-ε) complexity summaries in optimization [42]. These metrics provide an interpretable rate summary complementary to pass-rate metrics.

This framing follows empirical convergence-rate characterizations used in iterative optimization and search, including multiplicative (geometric-style) interpretations of residual reduction [41], while remaining descriptive rather than proving formal convergence guarantees. It is also consistent with iterative verifier-feedback refinement in code generation, where deterministic diagnostics guide successive corrections toward acceptance [17, 18].

3.5. Statistical Reliability Analysis

For convergence reliability, we model each prompt–model case as a Bernoulli success if it reaches eventual production-validator acceptance (zero SysIDE errors) within the allowed loop. Let p denote convergence probability for prompts drawn from the SysMBench-style benchmark distribution under the same controller, validator, and backend configuration. Because the validator-gated loop is monotonic and accepted cases terminate immediately, regression transitions (single-shot pass → pipeline fail) are structurally excluded. Improvement is therefore quantified descriptively rather than via symmetric paired hypothesis tests.

We report an exact 95% Clopper–Pearson lower confidence bound for p [43]; in the all-success case this is $p_L = (\alpha/2)^{1/n}$ with $\alpha = 0.05$. We also report the complementary one-sided 95% binomial upper confidence bound on failure probability $q = 1 - p$. In the zero-failure case, this bound is obtained by solving $(1 - q)^n = 0.05$, yielding $q_U = 1 - 0.05^{1/n}$. For large n, this expression is well approximated by $q_U \approx -\ln(0.05)/n \approx 3/n$.

These reliability bounds are intentionally scoped to SysMBench-style prompt distributions under the evaluated configuration and are reported numerically in Section 4. They are not treated as universal guarantees over arbitrary natural-language inputs.

3.6. Extended Analyses

In addition to the primary evaluation metrics described above, we conduct several supplementary analyses to better understand repair behavior within the validator-guided generation loop. These analyses are presented in Appendix B and Appendix C. Specifically, we examine relationships between iterations-to-success and generated output length as well as SysMBench benchmark difficulty labels (Appendix B). We also analyze persistent validator errors across repair iterations to characterize cases in which the language model fails to resolve a reported error despite attempting repair (Appendix C). These analyses provide additional insight into repair dynamics and factors influencing convergence behavior.

3.7. Reproducibility

All code, run artifacts, and analysis outputs used in this study are stored in the project repository [11]. The repository includes the scripts required to regenerate campaign statistics, tables, and figures.

4. RESULTS

In this section, we present the results of our evaluation of production validation outcomes across the benchmark. We begin by examining overall acceptance behavior and then analyze reliability, convergence dynamics, and validation characteristics across models.

4.1. Primary Outcome: Production Validation Acceptance

Across all 604 prompt-level trials (151 prompts for each of four models), single-shot production validation succeeded for 309/604 cases (51.16%). Under validator-in-the-loop refinement, pipeline production validation succeeded for 604/604 cases (100.00%).

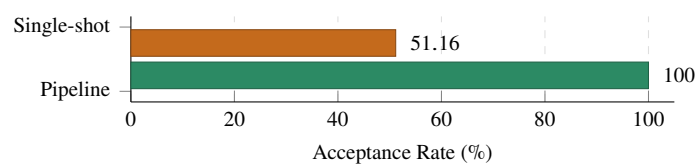

FIGURE 3: Overall single-shot vs final pipeline production-validation acceptance across all 604 prompt-level cases.

Copyright ©2026 by ASME

4.2. Per-Model Reliability

All models reached 151/151 eventual production validation under the validator-gated loop, but single-shot pass rates differed substantially. Figure 4 provides the per-model comparison with exact acceptance values annotated on the bars.

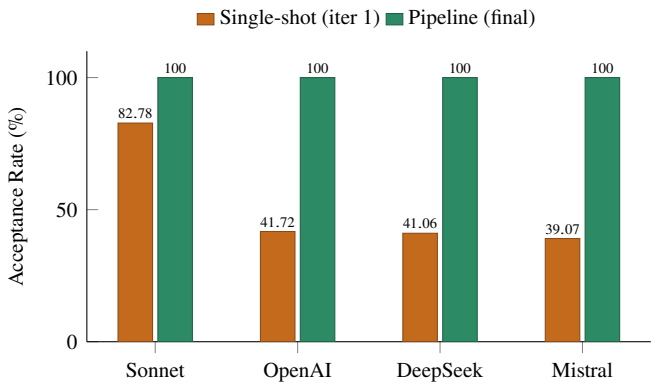

FIGURE 4: Per-model single-shot vs final pipeline production-validation acceptance.

Anthropic Sonnet 4.6 had the highest single-shot pass rate (82.78%), while OpenAI Codex 5.2 (41.72%), DeepSeek Reasoner (41.06%), and Mistral Large (39.07%) showed worse single-shot behavior and larger single-shot-to-pipeline gaps.

4.3. Convergence Behavior

Convergence is indexed by repair cycles: $k = 0$ denotes the initial single-shot generation (no validator feedback), and $k \geq 1$ denotes validator-guided repair cycles; acceptance corresponds to zero-error output under the production validator.

Figure 5 first presents the pooled convergence trajectory across all prompt-level cases.

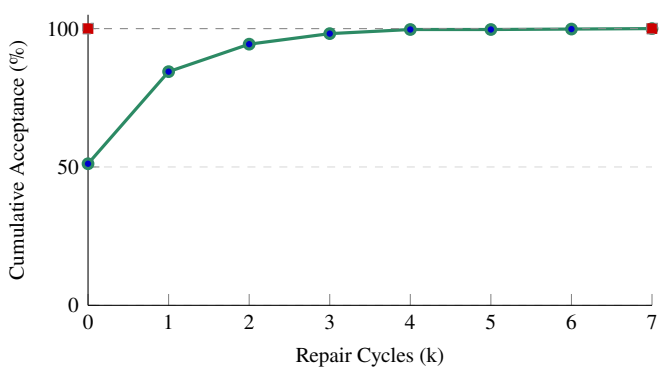

FIGURE 5: Cumulative production-validation acceptance versus repair cycles (k). $k = 0$ denotes initial single-shot generation; $k \geq 1$ denotes validator-guided repair cycles. Acceptance corresponds to zero-error output under the production validator.

The pooled curve shows a large first-step jump from $k = 0$ to $k = 1$, followed by rapid compression by $k = 2$ and then a short tail. Table 1 provides the exact counts and percentages underlying Figure 5: acceptance increases from 51.16% at $k = 0$ to 84.44% at $k = 1$, reaches 94.37% by $k = 2$, and reaches 99.67% by $k = 4$. Only two cases remain in the grouped $k = 5$–8 tail (0.33%), where cumulative acceptance reaches 100.00%. Consistent with this front-loaded pattern, iterations-to-acceptance are summarized by mean 1.727, median 1, IQR 1–2, and maximum 8.

TABLE 1: Distribution of repair cycles to first production-validation acceptance (pooled across 604 prompt-level cases).

Repair cycles (k)	Cases	Share	Cumulative
0	309	51.16%	51.16%
1	201	33.28%	84.44%
2	60	9.93%	94.37%
3	23	3.81%	98.18%
4	9	1.49%	99.67%
5–8	2	0.33%	100.00%

Rate Characterization. To quantify the observed convergence shape, we report an empirical contraction analysis of residual failure mass. Under contraction-style iteration analysis, let residual failure mass be $R_k = 1 - A_k$. The observed residual sequence is $R_0 = 0.4884$, $R_1 = 0.1556$, $R_2 = 0.0563$, $R_3 = 0.0182$, and $R_4 = 0.0033$. Using early cycles, the empirical contraction ratios are $\rho_0 \approx 0.1556/0.4884 \approx 0.32$, $\rho_1 \approx 0.0563/0.1556 \approx 0.36$, and $\rho_2 \approx 0.0182/0.0563 \approx 0.32$. These values cluster around an average early-cycle contraction factor of approximately 0.33 (computed as the arithmetic mean of ρ_0–ρ_2). Tail transitions are excluded from contraction-rate estimation because ratios become unstable when residual mass is near the finite-sample resolution. These early-cycle ratios cluster in a narrow band, suggesting an approximately multiplicative ("contraction-like") reduction pattern in the initial repair regime. In practical terms, early cycles remove roughly two-thirds of the remaining failures per cycle (in the observed early regime).

Complementing the contraction estimate, time-to-threshold metrics quantify convergence speed. The observed thresholds are $T_{90} = 2$, $T_{95} = 3$, and $T_{99} = 4$. These values characterize how rapidly validator-guided refinement approaches the acceptance set in repair-cycle space.

To assess whether this pooled behavior is shared across backends, Figure 6 overlays per-model cumulative acceptance trajectories.

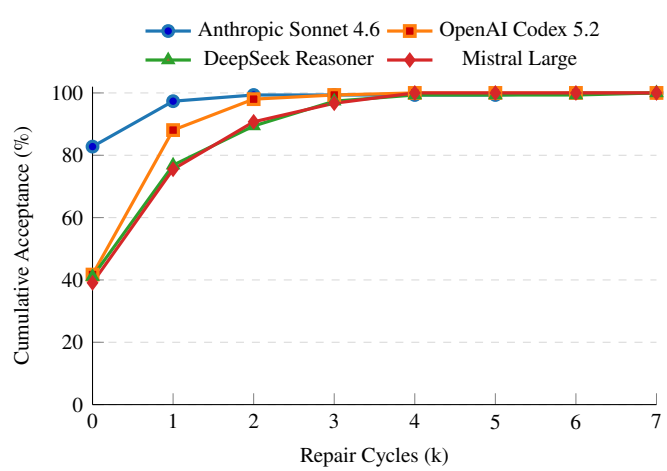

FIGURE 6: Per-model cumulative production-validation acceptance versus repair cycles (k).

Copyright ©2026 by ASME

Figure 6 shows that single-shot starting points differ across models, but the trajectories compress rapidly once validator-guided repair begins. Despite different initial acceptance levels, all curves move quickly toward full acceptance within a small number of repair cycles. This pattern supports the model-agnostic control-signal interpretation: deterministic validator diagnostics drive the dominant convergence dynamics across backends.

4.4. Statistical Reliability

We now quantify the reliability of validator-guided convergence across the full campaign. In total, we evaluated $n = 604$ prompt–model cases. In single-shot generation, 309 of 604 cases (51.16%) passed production validation. Under validator-guided refinement, all 604 cases ultimately reached zero-error production acceptance, with no observed failures.

To interpret this result statistically, we model convergence as a Bernoulli process and compute an exact 95% Clopper–Pearson lower confidence bound on the convergence probability p [43]. For the all-success case ($x = 604$ successes in $n = 604$ trials), the lower bound is

$$p_L = (\alpha/2)^{1/n},$$

with $\alpha = 0.05$. Substituting $n = 604$ gives

$$p_L = (0.025)^{1/604} \approx 0.9939.$$

With 95% confidence, the true convergence probability under this benchmark configuration is therefore at least 99.39%.

Equivalently, we can compute a one-sided 95% upper bound on the failure probability $q = 1 - p$ by solving $(1 - q)^n = 0.05$. For $n = 604$, this yields

$$q_U = 1 - 0.05^{1/604} \approx 0.00495,$$

or approximately 0.495%. A large-n approximation gives $q_U \approx -\ln(0.05)/604 \approx 0.00496$, which is consistent with the exact calculation.

In practical terms, this means that even though we observed zero failures in 604 cases, statistical uncertainty remains finite. Based on the binomial confidence analysis, we can state with 95% confidence that the true failure rate under this benchmark configuration is below approximately 0.5%, and equivalently that the true convergence probability exceeds 99%. While this does not imply perfect reliability, it indicates that production-validator convergence is highly probable for SysMBench-style prompts under the evaluated setup.

We emphasize that these bounds apply to SysMBench-style prompt distributions under the evaluated controller, validator, and backend configuration. They do not imply universal convergence over arbitrary natural-language inputs. At the same time, the absence of observed failures is consistent with the structure of the loop. Accepted cases terminate immediately, and rejected cases receive deterministic diagnostics that guide subsequent revisions. Within this benchmark distribution, validator-gated refinement exhibits stable and highly reliable convergence behavior.

4.5. Grammar Validity vs. Production Validity and Error Burden

In practice, a SysMLv2 artifact must pass production validation in a modeling environment before it can be used downstream. Grammar parsability alone is not sufficient. To quantify the gap between these two notions of validity, we compared ANTLR parsing outcomes against production validation outcomes for the initial candidate ($k = 0$) across all 604 prompt–model cases.

Of the 604 initial candidates, 369 (61.09%) passed grammar-level parsing, whereas only 309 (51.16%) passed production validation. In 60 cases, the generated model passed ANTLR parsing but failed production validation. These 60 cases represent 16.26% of grammar-valid artifacts (60/369) and 9.93% of all initial outputs (60/604). In other words, grammar-level parsing overestimates operational usability by approximately 16% among parseable artifacts. We did not observe any case in which a model passed production validation while failing grammar parsing (0/604), indicating that grammar validity is necessary but not sufficient for production acceptance under the evaluated configuration.

ANTLR \ SysIDE	Fail	Pass
Fail	235	0
Pass	**60**	309

FIGURE 7: Initial-candidate grammar (ANTLR) vs production validation (SysIDE), $n = 604$. The operational-gap cell (ANTLR pass, SysIDE fail) contains 60 cases.

4.6. Insights from Extended Analyses

First, we observe no meaningful relationship between iterations-to-success and either SysMBench difficulty or the length of the generated SysML output. Repair effort therefore does not appear to scale with model size or benchmark difficulty. One possible explanation is that raw line count does not capture the number of distinct grammatical structures that must be correctly formed; shorter outputs may still contain complex reference or typing relationships that trigger validator failures. Second, when validator errors persist across repair iterations, the majority occur because the LLM does not modify the validator-highlighted code region at all, effectively ignoring the reported diagnostic. In the remaining cases, the model appears to attempt a repair but produces another invalid construction. This pattern suggests both a pipeline efficiency gap, in which repair iterations are wasted when the highlighted region is not revised, and a model limitation, in which the model attempts a repair but fails to generate a syntactically valid correction.

4.7. Open-Source Dataset Release

We release the full trajectory-level corpus generated in this study rather than only baseline and final artifacts. The dataset spans the complete SysMBench prompt set of 151 prompts evaluated across four model backends, yielding 604 prompt–model

Copyright ©2026 by ASME

cases. For each case, we retain every validator-guided refinement step. In total, the release contains 1,043 stored iteration artifacts. Each artifact includes the generated SysMLv2 text and the corresponding production-validator diagnostics. As a result, the dataset preserves the full correction trajectory for each case, not just its endpoints.

At the artifact level, we classify iterations as positive if they pass production validation and negative if they fail. Under this definition, the release contains 604 positive artifacts and 439 negative artifacts. For a newly standardized language such as SysMLv2, where publicly available corpora of production-accepted models remain limited, these 604 positive artifacts provide a meaningful resource. The 439 negative artifacts correspond to intermediate repair states within the same 604 trajectories; they do not represent additional independent benchmark instances, but rather the correction path taken for each prompt–model pair.

Releasing full stepwise traces enables analyses that are not possible with endpoint-only datasets. Researchers can study repair dynamics under deterministic validator feedback, model convergence behavior across trajectories, and explore learning strategies that operate on intermediate states rather than final outputs alone. Because we retain runtime and token-level metadata at both iteration and trajectory levels, the dataset also supports systematic cost–reliability tradeoff analyses under a consistent production-validation criterion.

5. DISCUSSION

Production Validation: By placing a production validator inside the generation loop, we address a practical barrier to using natural-language–to–SysMLv2 generation in real MBSE workflows. Rather than producing text that appears structurally plausible, we require that every generated model satisfy the same production checks needed to load and use it in an industrial modeling environment. Across 604 prompt–model cases, validator-guided refinement increases production-validation acceptance from 51.16% in single-shot generation to 100.00%. In doing so, we remove the primary syntactic obstacle to integrating LLM-assisted modeling into existing toolchains.

Reliability Mechanism: Shifting from single-shot generation to validator-guided refinement also changes where reliability is enforced. In the single-shot setting, acceptance depends on whether the language model happens to produce a structurally correct artifact on its first attempt. In the validator-guided setting, we instead rely on a deterministic backend that evaluates and reports concrete modeling errors. At each iteration, we check the candidate model, revise it based on explicit diagnostics, and re-evaluate it until no validation errors remain. Because we apply the same production validator across all runs, acceptance becomes largely independent of backend-specific first-pass behavior. Model providers may differ in cost and latency, but we enforce production acceptance through a shared validation layer.

Deployment Implications: From a deployment perspective, we believe this distinction is critical. In typical MBSE environments, engineers must load, render, and validate models before using them for visualization, simulation, or verification. Without validator guidance, generated models are usable only when they happen to meet these constraints. With validator guidance, we enforce usability as part of the generation process itself. This allows us to incorporate LLM-assisted modeling into day-to-day engineering practice while maintaining bounded operational risk. In this setting, engineers can spend less time correcting structural syntax errors and more time reasoning about architecture, requirements, and verification intent.

Convergence Dynamics: The observed convergence behavior further supports this interpretation. Deterministic diagnostics provide stable correction signals, and we find that most recoverable failures resolve within a small number of repair cycles ($T_{90} = 2$, $T_{95} = 3$, $T_{99} = 4$). Initial candidates often contain multiple structural errors, yet the loop typically resolves these issues in only a few iterations. These results suggest that the refinement process corrects shallow but compound structural inconsistencies rather than exhibiting unstable or oscillatory dynamics.

Scope Limits: We emphasize that the scope of these results is intentionally limited. By enforcing production validation, we guarantee structural and operational compatibility with a modeling tool, but we do not guarantee that the resulting model faithfully represents the intended system or satisfies its requirements. We do not evaluate semantic correctness, behavioral adequacy, or trace completeness in this study. In addition, we bound our empirical findings to the SysMBench prompt distribution and to a single production validator backend.

Within these bounds, we view validator-driven refinement as a foundational reliability layer for AI-assisted MBSE. Once we guarantee structural acceptance, we can introduce additional mechanisms such as semantic validation, cross-view consistency analysis, simulation-based checks, and requirement trace evaluation. By establishing production-validator acceptance as a stable baseline, we create a controlled starting point for building higher-level assurances on top of syntactic convergence.

6. CONCLUSION

Our results indicate that single-shot LLM generation does not reliably produce SysMLv2 models that pass production validation, and this limits the direct use of natural-language–to–SysMLv2 pipelines in real MBSE workflows. In this work, we address this limitation by placing the production validator inside the generation loop. We generate a candidate model, run it through a production SysMLv2 validator, and revise it based on the reported diagnostics until the validator reports zero errors. In doing so, we treat production acceptance as the stopping condition of the generate–validate–repair process. Unlike grammar-level approaches that rely on parser checks alone, our method requires acceptance by an industrial modeling tool.

Across the full SysMBench prompt set of 151 prompts and four model backends, yielding 604 prompt–model cases, this approach increases production-validation acceptance from 51.16% in single-shot generation to 100.00% after validator-guided refinement. Most failures resolve within a small number of repair cycles.

From a practical standpoint, this work provides a reliability layer between LLM-generated text and MBSE tools. Instead of relying on a single pass of generation, we use deterministic validator feedback to drive correction until the model can be loaded

Copyright ©2026 by ASME

and used in a production environment. This approach does not require retraining model weights and remains agnostic to the underlying language model. To support further research, we release the full trajectory-level data from the campaign, comprising 1,043 iteration artifacts with associated validation traces and run metadata [11].

Our results are intentionally scoped to syntactic and operational acceptance under a production validator. Passing validation is necessary for deployment, but it does not guarantee semantic correctness or architectural adequacy. Future work should examine cross-validator replication, semantic alignment with requirements, and task-level evaluation of behavioral correctness. We view production-validator enforcement as a first step toward reliable NL-to-MBSE integration, not the final step.

7. LIMITATIONS AND FUTURE WORK

While the proposed validator-in-the-loop framework achieves production-validation acceptance across all benchmark cases, several limitations remain.

First, our study focuses exclusively on syntactic and production-level validation. Passing a production validator ensures that a model can be loaded, rendered, and processed by downstream tools, but it does not guarantee semantic correctness, architectural adequacy, or requirements fidelity. A model may satisfy all name resolution, typing, and ownership constraints while still misrepresenting the intended system behavior. Extending the framework to incorporate requirement-level checks, trace consistency, and behavior validation is an important next step.

Second, our experiments rely on a single production validator, SysIDE. Although SysIDE reflects industrial validation behavior, different toolchains may enforce additional constraints or differ in diagnostic reporting. Cross-validator replication would strengthen the generality of the approach and help characterize how validator differences affect repair trajectories and convergence behavior.

Third, the empirical results are scoped to the SysMBench prompt distribution. While the benchmark spans 151 prompts and multiple model backends, it does not capture the full variability of industrial natural-language specifications, including noisy, incomplete, or internally inconsistent requirements. Evaluating robustness under longer and less structured inputs would provide further insight into deployment readiness.

Fourth, although convergence is observed empirically in all 604 prompt–model cases, we do not provide a formal guarantee of convergence. The observed contraction-like behavior in early repair cycles suggests stable dynamics under deterministic diagnostics, but a theoretical analysis of convergence conditions and potential failure modes remains open.

Finally, we do not quantify cost–reliability tradeoffs in detail. Validator-guided refinement introduces additional model calls and runtime overhead. A systematic study of iteration counts, token usage, and latency under varying prompt complexity would clarify the practical operating envelope of the approach.

Future work will focus on three directions. First, we will extend the validator-in-the-loop paradigm to incorporate semantic and simulation-based checks, moving from syntactic acceptance to task-level correctness. Second, we will investigate cross-validator evaluation to assess toolchain sensitivity and strengthen claims of deployment robustness. Third, we will explore adaptive repair strategies that prioritize high-impact diagnostics to reduce iteration count and cost. Together, these efforts aim to build a layered assurance framework for reliable natural-language–to–MBSE generation.

REFERENCES

[1] Estefan, Jeff A. "Survey of Model-Based Systems Engineering (MBSE) Methodologies." Technical Report No. Rev. B. International Council on Systems Engineering (INCOSE), Seattle, WA, USA. 2008. URL https://www.omg.org/sysml/MBSE_Methodology_Survey_RevB.pdf. Prepared for the INCOSE MBSE Initiative.

[2] Madni, Azad M. and Sievers, Michael. "Model-based systems engineering: Motivation, current status, and research opportunities." *Systems Engineering* Vol. 21 No. 3 (2018): pp. 172–190. DOI 10.1002/sys.21438. Accessed 2026-03-13, URL https://incose.onlinelibrary.wiley.com/doi/10.1002/sys.21438.

[3] International Council on Systems Engineering (INCOSE). "Systems Engineering Vision 2035: Engineering Solutions for a Better World." Technical report no. INCOSE. 2021. URL https://www.incose.org/docs/default-source/aboutse/se-vision-2035.pdf. Accessed February 2026.

[4] Object Management Group. "OMG Systems Modeling Language (SysML) v2 Specification." Technical Report No. Version 1.0 Beta. Object Management Group (OMG). 2024. URL https://www.omg.org/spec/SysML/2.0/. Official specification draft, September 2024.

[5] Peng, Sida, Kalliamvakou, Eirini, Cihon, Peter and Demirer, Mert. "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot." (2023). DOI 10.48550/arXiv.2302.06590. Accessed 2026-03-13, URL http://arxiv.org/abs/2302.06590. ArXiv:2302.06590 [cs].

[6] Cui, Kevin Zheyuan, Peng, Sida, Quintana, Alexi and et al. "The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers." *Working Paper* (2025)URL https://economics.mit.edu/sites/default/files/inline-files/draft_copilot_experiments.pdf.

[7] Cui, Kevin Zheyuan, Demirer, Mert, Jaffe, Sonia, Musolff, Leon, Peng, Sida and Salz, Tobias. "The Productivity Effects of Generative AI: Evidence from a Field Experiment with GitHub Copilot." (2024). DOI 10.21428/e4baedd9.3ad85f1c. URL https://mit-genai.pubpub.org/pub/v5iixksv/release/2. MIT Generative AI Working Paper.

[8] Bouamra, Yasmine, Yun, Bruno, Poisson, Alexandre and Armetta, Frédéric. "SysTemp: A Multi-Agent System for Template-Based Generation of SysML v2." (2025). DOI 10.48550/arXiv.2506.21608. URL https://arxiv.org/abs/2506.21608. ArXiv:2506.21608.

[9] Cibrián, Eduardo, Olivert-Iserte, Jose, Llorens, Juan and Álvarez Rodríguez, Jose María. "An Agent-Based Approach for the Automatic Generation of Valid SysMLv2 Models in Industrial Contexts." *Computers in Industry* Vol. 172

Copyright ©2026 by ASME

(2025): p. 104350. DOI 10.1016/j.compind.2025.104350. URL https://www.sciencedirect.com/science/article/pii/S0166361525001150.

[10] Jin, Dongming, Jin, Zhi, Li, Linyu, Fang, Zheng, Li, Jia, Chen, Xiaohong and Luo, Yixing. "A System Model Generation Benchmark from Natural Language Requirements." *arXiv preprint arXiv:2508.03215* (2025)DOI 10.48550/arXiv.2508.03215. URL https://arxiv.org/abs/2508.03215.

[11] LaVoie, Chance, Andujar Lugo, Eladio and Kara, Levent Burak. "SysMBench Compiler-in-the-Loop Campaign Repository." https://github.com/cmuchancel/SysMBench_Compiler_In_Loop (2026). Code, artifacts, and analysis scripts for this study. Accessed February 21, 2026.

[12] Vaicenavičius, Juozas, Wiklund, Tilo, Kavolis, Daumantas, Draukšas, Simonas, Kalkauskas, Antanas and Vaicenavičius, Rimantas. "SysIDE: SysML v2 Textual Editing and Analysis System – Overview and Applications." *Workshop on Optimising MBSE Applications and Sustainability in Space Mission Design*. 2024. URL https://sensmetry.com/. Open-source SysML v2 textual modeling and validation environment.

[13] Clarke, Edmund M., Grumberg, Orna, Jha, Somesh, Lu, Yuan and Veith, Helmut. "Counterexample-Guided Abstraction Refinement." Emerson, E. Allen and Sistla, Aravinda Prasad (eds.). *Proceedings of the 12th International Conference on Computer Aided Verification (CAV 2000)*: pp. 154–169. 2000. Springer, Berlin, Heidelberg. DOI 10.1007/10722167_15.

[14] Solar-Lezama, Armando. "Program Synthesis by Sketching." Ph.D. Thesis, University of California, Berkeley. 2008. URL https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-176.html.

[15] Jha, Susmit, Gulwani, Sumit, Seshia, Sanjit A. and Tiwari, Ashish. "Oracle-Guided Component-Based Program Synthesis." *International Conference on Software Engineering (ICSE)*: pp. 215–224. 2010. DOI 10.1145/1806799.1806833.

[16] Alur, Rajeev, Bodik, Rastislav, Juniwal, Garvit, Martin, Milo M. K., Raghothaman, Mukund, Seshia, Sanjit A., Singh, Rishabh, Solar-Lezama, Armando, Torlak, Emina and Udupa, Abhishek. "Syntax-Guided Synthesis." *Proceedings of the 2013 Formal Methods in Computer-Aided Design (FMCAD)*: pp. 1–8. 2013. DOI 10.1109/FMCAD.2013.6679385. URL https://doi.org/10.1109/FMCAD.2013.6679385.

[17] Wang, Xin, Chen, Wenhu, Chen, Xinyun and Wang, William Yang. "Compilable Neural Code Generation with Compiler Feedback." *Findings of the Association for Computational Linguistics: ACL 2022*: pp. 138–150. 2022. URL https://aclanthology.org/2022.findings-acl.2/.

[18] Grubisic, Dejan, Cummins, Chris, Seeker, Volker and Leather, Hugh. "Compiler Generated Feedback for Large Language Models." *arXiv preprint arXiv:2403.14714* (2024)DOI 10.48550/arXiv.2403.14714. URL https://arxiv.org/abs/2403.14714.

[19] Wang, Yuan, Ge, Ning, Liu, Jiangxi, Cao, Zhilong, Chen, Zheping and Hu, Chunming. "Generating SysML Behavior Models via Large Language Models: An Empirical Study." *Proceedings of the 16th International Conference on Internetware (Internetware 2025)*. 2025. ACM. DOI 10.1145/3755881.3755926. URL https://dl.acm.org/doi/10.1145/3755881.3755926.

[20] DeHart, John K. "Leveraging Large Language Models for Direct Interaction with SysML v2." *INCOSE International Symposium* Vol. 34 No. 1 (2024): pp. 2168–2185. DOI 10.1002/iis2.13262. URL https://doi.org/10.1002/iis2.13262.

[21] Yang, Bao, Yang, Zhibin, Yang, Yongqiang, Xie, Jian, Zhou, Yong, Yue, Tao, Huang, Zhiqiu and Guo, Peng. "An Automated Approach to Generate SysML Models from Restricted Natural Language Requirements in Chinese." *Journal of Computer Research and Development* Vol. 58 No. 4 (2021): pp. 706–730. DOI 10.7544/issn1000-1239.2021.20200757.

[22] Zhong, Shaohong, Scarinci, Andrea and Cicirello, Alice. "Natural Language Processing for Systems Engineering: Automatic Generation of Systems Modelling Language Diagrams." *Knowledge-Based Systems* Vol. 259 (2023): p. 110071. DOI 10.1016/j.knosys.2022.110071. URL https://doi.org/10.1016/j.knosys.2022.110071.

[23] Mistry, Vivek et al. "Text-to-Model Transformation: Natural Language-Based Model Generation for SysML Diagrams." *Systems* Vol. 12 No. 9 (2024): p. 369. DOI 10.3390/systems12090369. URL https://www.mdpi.com/2079-8954/12/9/369.

[24] Authors, Anonymous. "LLM-Assisted Semantic Alignment and Integration in Collaborative SysML v2 Modeling." *arXiv preprint arXiv:2508.16181* (2025)URL https://arxiv.org/abs/2508.16181.

[25] Hendricks, Matthew Anderson and Cicirello, Alice. "Text to Model via SysML: Automated Generation of Dynamical System Computational Models from Unstructured Natural Language Text." *arXiv preprint arXiv:2507.06803* (2025)URL https://arxiv.org/abs/2507.06803.

[26] Geng, Saibo, Josifoski, Martin, Peyrard, Maxime and West, Robert. "Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning." *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2023. URL https://aclanthology.org/2023.emnlp-main.674/.

[27] Park, Kanghee, Wang, Jiayu, Berg-Kirkpatrick, Taylor, Polikarpova, Nadia and D'Antoni, Loris. "Grammar-Aligned Decoding." *Advances in Neural Information Processing Systems (NeurIPS 2024)*. 2024. URL https://arxiv.org/abs/2405.21047.

[28] Garcia, Carlos, Castro, Isabel and Johnson, Mark. "Grammar-Constrained Decoding for Structured Information Extraction with Low-Resource Transformers." *Frontiers in Artificial Intelligence* Vol. 7 (2024). DOI 10.3389/frai.2024.1406857. URL https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2024.1406857/full.

Copyright ©2026 by ASME

[29] Ravi, Ravin, Bradshaw, Dylan, Ruberto, Stefano, Jahangirova, Gunel and Terragni, Valerio. "LLMLOOP: Improving LLM-Generated Code and Tests through Automated Iterative Feedback Loops." *IEEE International Conference on Software Maintenance and Evolution (ICSME 2025)*. 2025. IEEE. URL https://valerio-terragni.github.io/assets/pdf/ravi-icsme-2025.pdf.

[30] Sevenhuijsen, Merlijn, Etemadi, Khashayar and Nyberg, Mattias. "VeCoGen: Automating Generation of Formally Verified C Code with Large Language Models." *arXiv preprint arXiv:2411.19275* (2024)URL https://arxiv.org/abs/2411.19275.

[31] Morvalho, Pedro M. et al. "Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization." *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI 2025)*. 2025. URL https://arxiv.org/abs/2502.07786.

[32] Namin, Javid et al. "VerMCTS: Synthesizing Multi-Step Programs using a Verifier, a Large Language Model, and Tree Search." *arXiv preprint arXiv:2402.08147* (2024)URL https://arxiv.org/abs/2402.08147.

[33] Sireum Project. "SysML v2 Parser for HAMR." https://github.com/sireum/hamr-sysml-parser (2026). GitHub repository `sireum/hamr-sysml-parser`; commit `d7c87942ca9f84de611415c8cca0c5916cc9ccae`; Accessed February 22, 2026.

[34] Systems Modeling Community. "SysML v2 Pilot Implementation." https://github.com/Systems-Modeling/SysML-v2-Pilot-Implementation (2026). GitHub repository `Systems-Modeling/SysML-v2-Pilot-Implementation`; commit `a5a602d28c570cfe1bb191041a1125f64d917a36`; Accessed February 22, 2026.

[35] OpenAI. "GPT-5.2-Codex Model Documentation." https://developers.openai.com/api/docs/models/gpt-5.2-codex (2026). Model used in this campaign: `gpt-5.2-codex`. Accessed February 20, 2026.

[36] Anthropic. "Introducing Claude Sonnet 4.6." https://www.anthropic.com/news/claude-sonnet-4-6 (2026). API usage specifies `claude-sonnet-4-6`. Accessed February 20, 2026.

[37] DeepSeek. "Reasoning Model (`deepseek-reasoner`)." https://api-docs.deepseek.com/guides/reasoning_model (2026). Accessed February 20, 2026.

[38] Mistral AI. "Function Calling (Model Example: `mistral-large-latest`)." https://docs.mistral.ai/capabilities/function_calling/ (2026). Accessed February 20, 2026.

[39] Banach, Stefan. "Sur les Opérations dans les Ensembles Abstraits et Leur Application aux Équations Intégrales." *Fundamenta Mathematicae* Vol. 3 No. 1 (1922): pp. 133–181. DOI 10.4064/fm-3-1-133-181. URL https://eudml.org/doc/213289.

[40] Nocedal, Jorge and Wright, Stephen J. *Numerical Optimization*, 2nd ed. Springer, New York, NY, USA (2006). DOI 10.1007/978-0-387-40065-5. URL https://doi.org/10.1007/978-0-387-40065-5.

[41] He, Jiawei and Lin, Guangming. "Average Convergence Rate of Evolutionary Algorithms." *IEEE Transactions on Evolutionary Computation* Vol. 20 No. 2 (2016): pp. 316–321. DOI 10.1109/TEVC.2015.2444793. URL https://doi.org/10.1109/TEVC.2015.2444793.

[42] Polyak, Boris T. *Introduction to Optimization*. Optimization Software, Inc., New York, NY, USA (1987).

[43] Clopper, C. J. and Pearson, E. S. "The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial." *Biometrika* Vol. 26 No. 4 (1934): pp. 404–413. DOI 10.1093/biomet/26.4.404. URL https://doi.org/10.1093/biomet/26.4.404.

APPENDIX A. AUXILIARY DEMONSTRATION: GRAMMAR PARSABILITY VS. PRODUCTION VALIDATION

To support the design choice of using production validation (rather than grammar parsing alone) as the acceptance oracle, we ran an auxiliary demonstration included in the repository [11]. This demonstration is not a second primary experiment; it is a construct-validity check showing that parser acceptance and production acceptance are distinct outcomes.

We evaluated 10 intentionally distinct SysMLv2 examples in `examples/mismatch_10_distinct/`. Each file was designed to remain grammar-parseable while violating a model-wide constraint typically enforced by a production validator. The evaluation pipeline was:

1. ANTLR parse check using the HAMR SysML v2 ANTLR4 parser (generated grammar derived from the SysML v2 Pilot Implementation) [33, 34].

2. Production validation check using SysIDE.

Results were unambiguous: all 10/10 examples passed ANTLR parsing, while 0/10 were accepted by production validation (10/10 mismatch cases). Validator diagnostics were dominated by unresolved-reference failures (9/10, `reference-error`), with one invocation-typing failure (1/10, `invocation-expression-instantiated-type`). This pattern directly illustrates that context-free syntax conformance is necessary but not sufficient for operational model acceptance in production modeling environments.

Concrete Example

The following example is grammar-parseable but fails production validation:

```
package Demo07 {
part def Wheel {
attribute hubDiameter: LengthValue;
part tire { attribute width: LengthValue; }
attribute outerDiameter: LengthValue = (hub-
Diameter + 2 * tire.height);
}
}
```

ANTLR parsing accepts the structure. Production validation rejects it with:

Copyright ©2026 by ASME

```
error (reference-error): No Feature named
'height' found.
```

TABLE 2: Ten-case auxiliary demonstration: all examples parse under ANTLR, none pass production validation.

ID	Injected Condition	ANTLR	SysIDE
01	Missing imported namespace	Pass	Fail (reference)
02	Missing port type	Pass	Fail (reference)
03	Missing attribute type	Pass	Fail (reference)
04	Missing specialization base	Pass	Fail (reference)
05	Action typed by undefined behavior type	Pass	Fail (reference)
06	Invocation target is not a behavior	Pass	Fail (invocation)
07	Missing referenced feature in expression	Pass	Fail (reference)
08	Missing root-qualified namespace	Pass	Fail (reference)
09	Redefinition of missing feature	Pass	Fail (reference)
10	Missing namespace in type use	Pass	Fail (reference)

These appendix additions summarize selected follow-on analyses over 151 SysMBench prompts and four model backends (604 total model-prompt runs). Because eventual syntactic success reaches 100% under iterative repair in this campaign, the more informative appendix-level outcomes are iterations-to-success and adjacent-iteration repair behavior.

APPENDIX B. EXTENDED ANALYSIS: DIFFICULTY AND OUTPUT-LENGTH SIGNALS

This appendix section examines whether iteration count is meaningfully related to either the SysMBench difficulty label or the length of the generated SysML output. The purpose is to test whether repair effort is primarily a size-driven phenomenon.

B.1. SysMBench Difficulty Versus Iterations-to-Success

In this first analysis, we use the SysMBench difficulty classification [10]. In their scheme, each prompt is assigned a difficulty bucket from the line count of the hand-authored ground truth SysML model, not from the generated output. The bucket definitions are: difficulty 1 for fewer than 30 lines, difficulty 2 for 30–59 lines, difficulty 3 for 60–89 lines, difficulty 4 for 90–119 lines, and difficulty 5 for 120 or more lines. Across the 151 benchmark prompts, this yields $n = 63, 64, 12, 6$, and 6 prompts in difficulty buckets 1 through 5, respectively. Because each prompt is evaluated with four models, the pooled right-panel averages in Figure 8 are computed from 252, 256, 48, 24, and 24 prompt-model cases across the five difficulty levels. Figure 8 tests whether that benchmark-defined size proxy tracks repair effort in the validator-in-the-loop setting.

The pooled mean iterations-to-success are 1.766, 1.668, 1.938, 1.667, and 1.583 across difficulty levels 1 through 5, respectively. The weak linear fit indicates that the hand-authored ground truth SysML line-count difficulty label is not strongly correlated with repair effort, as measured here by iterations-to-success.

B.2. Generated Output Length Versus Iterations-to-Converge

Since SysMBench difficulty is line-count based, and one might reasonably expect repair effort to increase with the number of lines generated, it is useful to ask whether the length of the *generated* SysML output is itself correlated with iterations-to-converge.

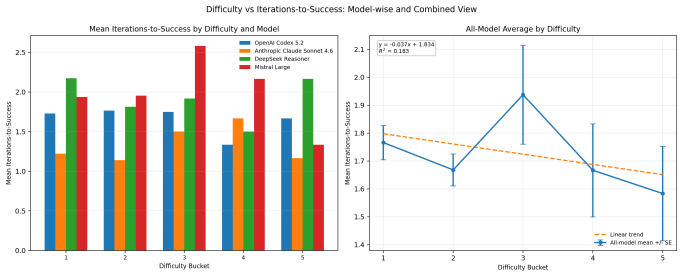

FIGURE 8: Difficulty-conditioned repair effort with model split and pooled trend. The pooled mean iterations-to-success are 1.766, 1.668, 1.938, 1.667, and 1.583 for difficulty levels 1 through 5, respectively, with a pooled linear fit of $R^2 \approx 0.183$.

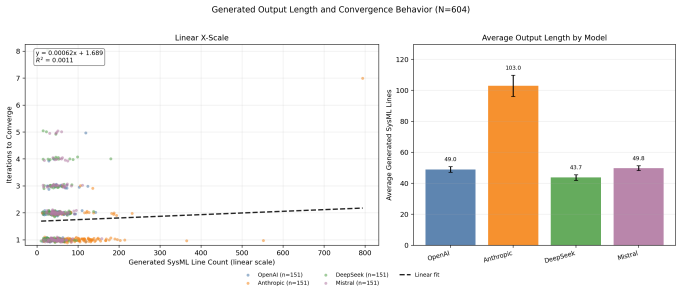

FIGURE 9: Generated SysML output length analysis. The left panel shows a scatter plot of generated SysML line count versus iterations-to-converge, with the pooled linear fit overlaid (R^2 = 0.0011, slope = 0.00062). The right panel shows the average generated SysML line count for each model, with error bars indicating standard error.

The pooled linear fit in Figure 9 is essentially flat (R^2 = 0.0011, slope = 0.00062), which indicates that generated SysML line count explains almost none of the variation in iterations-to-converge. In this analysis, generated output length is therefore not strongly correlated with repair effort.

The model-level averages in the right panel reinforce the same point. OpenAI, DeepSeek, and Mistral produce broadly similar output lengths on average (approximately 49, 44, and 50 lines, respectively), whereas Anthropic produces about 103 lines on average, or roughly twice as many. However, Anthropic also required the fewest iterations-to-success on average in the main results (1.23, versus 1.73 for OpenAI, 1.97 for DeepSeek, and 1.98 for Mistral), further indicating that line count alone is a poor standalone proxy for repair effort. Taken together, Figures 8 and 9 suggest that repair effort is not well explained by size alone.

One possible explanation is that raw line count does not necessarily reflect the number of distinct syntactic structures that must be formed correctly. A model can generate many lines by expanding a relatively uniform pattern, whereas a shorter output may still involve a wider variety of declarations, references, and nested relationships that are more sensitive to validator failure. Simply producing more lines of the same structure therefore need not imply greater repair effort.

Copyright ©2026 by ASME

APPENDIX C. EXTENDED ANALYSIS: CLASSIFYING REPAIR ATTEMPTS ON PERSISTENT ERRORS

This appendix section looks at what the LLM does from one iteration to the next when a persistent error is still present. We treat an error as *persistent* if an identical SysIDE diagnostic, meaning the same error family and the same `stderr` message, appears in two consecutive iterations of the same prompt-model run. To study the repair attempt itself, we compare the SysML artifact from iteration k, which is the previous attempt fed back into the loop, with the revised SysML artifact produced by the LLM at iteration $k + 1$.

C.1. Error Transition Outcome Explanations

Using the SysIDE `stderr` outputs, we ask a simple question: when an identical error is still present at iteration $k + 1$, did the LLM appear to change the highlighted failing region, or did it leave that region unchanged? Across all back-to-back iteration pairs, this yields 1502 exact-error transitions. Each transition corresponds to one exact SysIDE diagnostic observed at iteration k and checked again at iteration $k + 1$. This is smaller than the total number of raw error instances because repeated copies of an identical SysIDE message within a single iteration are collapsed into one signature-level transition event. That choice is intentional: here we want to measure whether the model corrected an error pattern from one iteration to the next, not whether it repaired every repeated occurrence independently. We interpret many repeated identical grammar diagnostics as manifestations of the same underlying structural issue, so once the model applies the relevant pattern-level fix, repeated instances will often be corrected together rather than one by one.

Of these 1502 transitions, 1262 are resolved by the next iteration and are therefore not persistent. The remaining 240 persist through the next repair attempt. These 240 persistent transitions are the cases for which we want to understand whether the model tried to repair the highlighted region but failed, or whether it effectively carried the same failing code forward unchanged.

We then classify the 240 persistent cases into one of two categories:

- **Unaddressed and not fixed**: an identical validator error remains at iteration $k + 1$, and the validator highlights an identical code snippet again.
- **Addressed but not fixed**: an identical validator error remains at iteration $k + 1$, but the validator now highlights a different code snippet instead, suggesting that the LLM changed the code to attempt to fix the error, but was unsuccessful.

To classify these persistent cases, we use the `stderr` outputs from SysIDE directly. For each persistent identical validator error, we compare the SysIDE `stderr` block at iteration k with the corresponding `stderr` block at iteration $k + 1$, and check whether SysIDE is still highlighting the identical code snippet or now highlights a different snippet instead.

For example, consider the following SysIDE `stderr` output. For readability, the long expected-token list is abbreviated here, but the full raw message is used in the analysis:

```
iteration_01.sysml:36:19: error (parsing-error):
    Unexpected 'state', expected one of [...]
    36 |         attribute state : VehicleState;
       |                   ^^^^^
```

In this example, we classify the transition as "unaddressed and not fixed" only if iteration $k + 1$ still contains that identical `parsing-error` message and still highlights the identical code snippet, `attribute state : VehicleState;`. If iteration $k + 1$ still contains the identical `parsing-error` message but SysIDE now highlights a different code snippet instead, we classify the transition as "addressed but not fixed." This is determined directly from validator stderr rather than by manually reading the full file.

A small amount of noise is possible for the "addressed but not fixed" category if multiple identical errors occur in different code regions within the same file. In that edge case, the LLM could fix one identical occurrence but fail to fix another identical occurrence on a different code segment, which would still appear here as "addressed but not fixed." We expect this to be uncommon in practice, because many grammar repairs are pattern-level corrections that tend to fix repeated identical structures together.

C.2. Persistent Error Analysis

Figure 10 restricts attention to the 240 persistent transitions and shows the split between the two persistent outcomes. Of these persistent cases, 155/240 (64.6%) are "unaddressed and not fixed," while 85/240 (35.4%) are "addressed but not fixed."

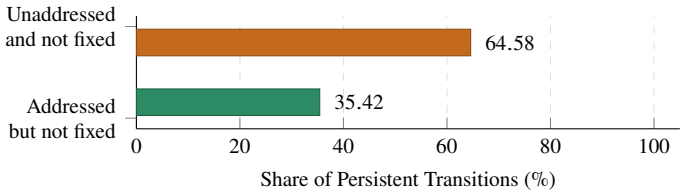

FIGURE 10: Persistent exact-error outcomes among the 240 transitions for which an identical validator error remains present at iteration $k + 1$. Of these persistent cases, 155/240 (64.6%) are unaddressed and not fixed, while 85/240 (35.4%) are addressed but not fixed.

The larger unaddressed-and-not-fixed share points to an inefficiency in the repair loop. In these cases, SysIDE is still highlighting the identical offending code region in the next iteration, suggesting that the LLM did not change that highlighted region after receiving the validator feedback. This motivates future work on stronger grounding of the prompt to the validator-highlighted snippet, or repair policies that explicitly require the highlighted region to be revised before resubmission.

The addressed-but-not-fixed share instead suggests LLM limitations. In these cases, the model appears to make a repair attempt, but the identical validator error still remains. This suggests a limitation in repair capability rather than simple loop inefficiency: the model changes the code, but does not produce a syntactically valid correction. Future work here should focus on constrained editing or fine-tuning with the large datasets of syntactically valid code that can now be produced. One caveat is that

Copyright ©2026 by ASME

this analysis is based on SysIDE `stderr` and highlighted snippets, so rare repeated-identical-error cases can still add a small amount of noise.

Copyright ©2026 by ASME